SendFlow: deeper product and implementation review

With the additional documents — **03 · Recorridos (App Flow)** and **06 · Plan de implementación** — I can be more concrete about what I meant by “validate the whole ecosystem.”

The documents describe a product with three connected paths:

1. **SendFlow Bike:** capture, import, analyze, and store riding data.
2. **SendFlow Coach:** connect riders and coaches, exchange footage and telemetry, review runs, and manage training plans.
3. **SendFlow Lab:** eventually connect suspension testing and servicing to a rider’s authorized history.

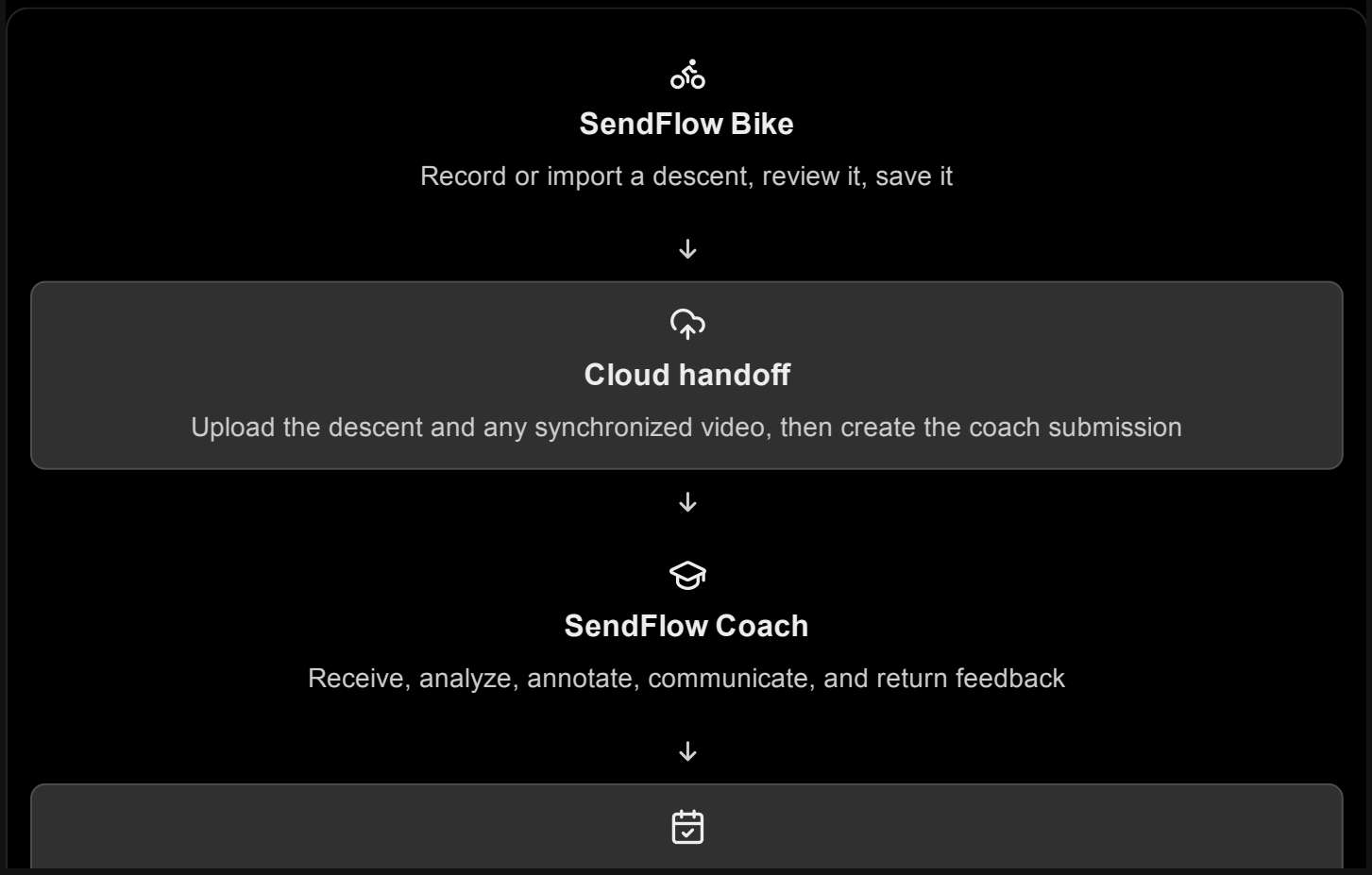
The implementation plan identifies four major blockers: assembling the real Sender and recording the first real descent, verifying Coach end to end in production, unifying descent data in the cloud, and deciding the FlowBench actuator.

06_IMPLEMENTATION_PLAN.md

My main recommendation is to **treat these as four distinct engineering risks, not one general “finish the app” task**. Each needs its own evidence of completion.

1. The highest-risk area is the boundary between products

The app flow lays out a specific journey:



Training follow-through

Coach creates a plan; rider follows sessions and reports completion

This is a sensible architecture from the user's perspective. But the boundaries are where failures can hide: the upload can succeed while the submission fails, the video can arrive without its descent data, or the coach can finish an analysis without the rider receiving the correct notification.

Your flow document already calls for retrying interrupted uploads and retaining a visible “sin subir” state when an upload cannot complete. It also specifies that a descent sent twice must not be duplicated in the unified cloud model.

03_APP_FLOW.md

06_IMPLEMENTATION_PLAN.md

I would make this the first formal integration test.

2. Four risks I would investigate first

Risk 1 **Production handoff and cloud consistency**

The plan calls for one canonical cloud representation of a descent, migrating existing records, moving storage to R2, and ensuring that Bike and Coach display the same descent.

06_IMPLEMENTATION_PLAN.md

What I would test:

- Upload one descent with video, one with telemetry only, and one with both.
- Interrupt an upload and retry it.
- Upload the same descent twice.
- Delete a descent and confirm that its associated media and database references behave as intended.
- Open the same submission from the rider and coach accounts and confirm that both see consistent metadata and status.

Completion evidence: a test report with expected versus actual results, including the database record and storage object—not just screenshots of successful screens.

Risk 2 **Real Sender data**

The implementation plan says the Bike app has been tested with a demo and simulated Sender, but has never seen a real descent. It also lists unfinished firmware endpoints and sensor drivers.

06_IMPLEMENTATION_PLAN.md

Why this matters: simulated data can validate the interface and file parser, but it cannot validate sensor noise, mechanical installation, real-world timing, calibration, power behavior, or whether a physical recording survives the complete pipeline.

What I would test: first a bench recording, then a real descent, then a descent with GoPro synchronization, and finally the same workflow on the supported phones and tablet.

The plan sets a gate of at least 10 real descents across two tracks before moving into the hardware-dependent phase. I would preserve that gate.

06_IMPLEMENTATION_PLAN.md

Risk 3 **Long-video reliability**

The Coach flow allows videos up to 1 GB, browser-side compression, chunked uploads, retries, and resumption.

03_APP_FLOW.md

03_APP_FLOW.md

This is a meaningful engineering requirement, not just a file-picker feature.

What I would test:

- Upload on a slow mobile connection.
- Leave the tab, return to it, and interrupt the network.
- Test a video that cannot be compressed in the browser.
- Test low device memory and unsupported video formats.
- Confirm that the displayed progress matches the actual transfer state.
- Confirm that a failed upload can be retried without leaving orphaned or duplicate files.

The specification should distinguish a recoverable interruption from a permanent failure and state exactly what the user can do in each case.

Risk 4 **Identity, invitations, and authorization**

Bike and Coach use the same account and database, while Coach can also register external trainers who do not own a Sender. The flow includes manual account approval, invitations, coach–rider relationships, and shared analyses.

03_APP_FLOW.md

What I would test:

- A coach cannot read another coach's private submissions.
- A rider can only share a descent with an authorized coach.
- Removing a coach revokes future access where required.

- An expired invitation cannot be reused.
- A logged-out user cannot access a bookmarked analysis URL.
- Switching between Bike and Coach does not accidentally inherit the wrong account or role.

Important: hiding a button is not authorization. The database and server-side functions must independently enforce access rules.

3. Your implementation plan needs a status reconciliation

This is one of the clearest improvements I would make before more development.

The implementation plan's opening status says Coach v28 is ready to upload, and its status section lists database scripts `coach_v10` through `coach_v13` as pending. However, Phase 0 still describes the older target of Coach v21 and Bike v25, and its checklist references migrations only through `coach_v9`. Phase 4 also says the training-plan feature was already advanced in Coach v26, while the initial Phase 0 completion criteria still reflect the earlier product state.







That does not necessarily mean the software is wrong. It means the plan may no longer reliably tell a new developer which work remains.

I would reconcile it into a single current-state table:

Workstream	Current evidence needed	Exit condition
Bike production	Exact deployed version and smoke-test report	Core flows pass in production
Coach production	Exact deployed version and two-account test	Rider-to-coach-to-rider loop passes
Database	Migration ledger and verified schema	All required migrations applied in order
Unified descent storage	Migration report and duplicate-upload test	One canonical record and correct media links
Sender hardware	Assembly checklist and bench log	Stable physical recordings
Training plans	Real coach and rider pilot	One real week completed and reviewed

I would also separate status into three independent fields:

- **Implementation:** not started, in progress, implemented.

- **Deployment:** local, staging, production.
- **Validation:** simulated, bench-tested, real-device-tested, pilot-validated.

That avoids treating “implemented” as equivalent to “ready for customers.”

4. The training-plan feature deserves its own validation

The app flow defines a specific manual workflow: the coach creates a plan for a linked athlete, sets its dates and phases, adds sessions to the week, and can copy a week forward. The athlete opens a session and records whether they completed it, their perceived effort, and an optional note.

03_APP_FLOW.md

I would not redesign this workflow yet. I would test whether it is dependable enough for a coach to use with real athletes.

Plan integrity

Can a coach edit or copy a week without unintentionally overwriting another week or changing completed sessions?

Calendar correctness

Are session dates, race dates, rest days, and time zones consistent between the coach and athlete views?

Completion history

Can the athlete revise an accidental response? Does the coach see the latest status without losing the original context?

Real coaching use

Can a coach build a representative week quickly, and can an athlete understand what to do each day without additional explanation?

The implementation plan already sets a useful milestone: a real coach creates a plan and an athlete follows it for one week. I would use that as the acceptance test before expanding the planning feature.

06_IMPLEMENTATION_PLAN.md

I would also make sure the product distinguishes a *planned session* from a *completed session*. They are different records conceptually, and their history should remain interpretable when plans change.

5. A practical two-week validation sprint

Rather than trying to finish every phase simultaneously, I would run this as a focused validation sprint. This is my proposed schedule, not a timeline already committed to in your documents.

Week 1 — Production and integration

1. Reconcile the deployed versions, pending migrations, secrets, and production configuration.
2. Run the complete two-account Coach flow: invitation, submission, analysis, feedback, and rider review.
3. Test large uploads, interrupted transfers, duplicate submissions, and authorization boundaries.
4. Document every failure with reproduction steps, severity, and an assigned fix.

Week 2 — Real-user validation

1. Have a coach and rider use the training-plan feature for a representative week.
2. Validate the unified descent data path against the real schema and storage behavior.
3. Complete the Sender bench test if the hardware is ready; otherwise keep hardware validation as an explicit separate blocker.
4. Re-run failed tests and publish a release-readiness checklist with evidence.

6. What I would leave alone for now

Your implementation plan says not to add anything outside the PRD without approval, and it explicitly keeps FlowBench/Lab as a separate development track.

06_IMPLEMENTATION_PLAN.md

I

agree

with

that

discipline.

For now, I would avoid:

- Introducing AI-generated training plans when the requirements specify coach-authored plans.
- Expanding the Coach feature list before the existing upload and feedback workflow is reliable.
- Treating simulated telemetry as evidence that the physical Sender is validated.
- Starting Lab software development before the actuator decision and mechanical requirements are resolved.

- Reworking already-defined user flows without evidence from real users.

The goal is to finish and validate the system you specified—not accidentally turn the project into several additional products.

7. What I need to give you a true implementation audit

The documents describe intended behavior and project status. They do not let me verify the current production code, database policies, or live behavior by themselves.

The most useful next artifact would be **the Coach source or its current builder project**, particularly `construir_coach.py`, together with the relevant SQL migrations. The implementation plan identifies these as the sources used to generate and configure the product.

06_IMPLEMENTATION_PLAN.md

With those files, I could inspect:

- Whether the app implements the specified flows.
- Whether database permissions enforce the intended ownership model.
- Whether upload and retry behavior is robust.
- Whether migration order and current schema agree.
- Which requirements are missing, incomplete, or implemented differently from the PRD.

Please exclude `.env` files, service-role keys, signing keys, passwords, and other secrets.

My bottom line: SendFlow's next milestone should be a verified, end-to-end Coach workflow backed by consistent cloud data, followed by validation with a real coach and rider. In parallel, the Sender needs its own hardware-validation track. Those are the two strongest paths toward proving the product works beyond its current simulated and implementation-stage state.